Cheat Sheet

Extended Slicing and String Methods

Extended Slicing

It is an extension of slicing that you have learnt.

Syntax:

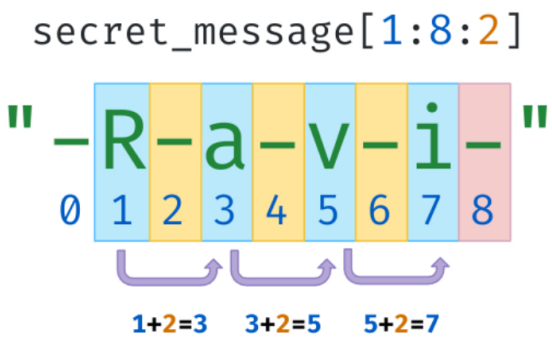
`variable[start_index:end_index:step]`

Along with `start_index` , and `end_index` we will be providing `step` . The `step` determines the increment between each index for slicing.

Code:

PYTHON

```
secret_message = "-R-a-v-i-"
print(secret_message[1:8:2])
```



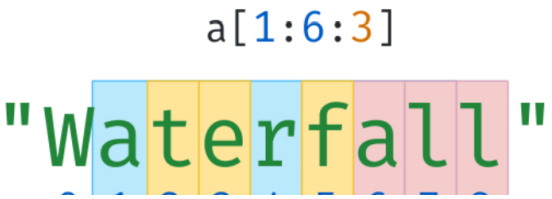
Output

```
Ravi
```

Code

PYTHON

```
a = "Waterfall"
part = a[1:6:3]
print(part)
```



Python 3.10

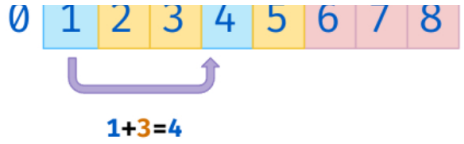
DEBUG RUN CODE

Your Input

Your Input

Your Input

Your Input



Output

```
ar
```

Methods

Python has a set of built-in reusable utilities.
They simplify the most commonly performed operations are:

String Methods

- `isdigit()`
- `strip()`
- `lower()`
- `upper()`
- `startswith()`
- `endswith()`
- `replace()` and more...

Isdigit

Syntax: `str_var.isdigit()`

Gives `True` if all the characters are digits. Otherwise, `False`

Code

PYTHON

```python
is_digit = "4748".isdigit()
print(is_digit)
```

Output

```
True
```

Strip

Syntax: `str_var.strip()`

Removes all the leading and trailing spaces from a string.

Code

PYTHON

```python
mobile = " 9876543210 "
mobile = mobile.strip()
print(mobile)
```

Output

```
9876543210
```

Strip - Specific characters

Your Input

Your Input

Your Input

Your Input

Your Input

Your Input

Syntax: `str_var.strip(chars)`

We can also specify characters that need to be removed.

Code

PYTHON

```python
name = "Ravi."
name = name.strip(".")
print(name)
```

Output

```
Ravi
```

Strip - Multiple Characters

Removes all spaces, comma(,) and full stop(.) that lead or trail the string.

Code

PYTHON

```python
name = ", .. ,, ravi ,, .. ."
name = name.strip(" ,.")
print(name)
```

Output

```
ravi
```

Replace

Syntax: `str_var.replace(old,new)`

Gives a new string after replacing all the occurrences of the old substring with the new substring.

Code

PYTHON

```python
sentence = "teh cat and teh dog"
sentence = sentence.replace("teh","the")
print(sentence)
```

Output

```
the cat and the dog
```

Startswith

Syntax: `str_var.startswith(value)`

Gives `True` if the string starts with the specified value. Otherwise, `False`

Code

PYTHON

```python
url = "https://onthegomodel.com"
is_secure_url = url.startswith("https://")
print(is_secure_url)
```

Output

Your Input

Your Input

Your Input

Your Input

Your Input

```
True
```

Endswith

Syntax: `str_var.endswith(value)`

Gives `True` if the string ends with the specified value. Otherwise, `False`

Code

PYTHON

```python
gmail_id = "rahul123@gmail.com"
is_gmail = gmail_id.endswith("@gmail.com")
print(is_gmail)
```

Output

```
True
```

Upper

Syntax: `str_var.upper()`

Gives a new string by converting each character of the given string to uppercase.

Code

PYTHON

```python
name = "ravi"
print(name.upper())
```

Output

```
RAVI
```

Lower

Syntax: `str_var.lower()`

Gives a new string by converting each character of the given string to lowercase.

Code

PYTHON

```python
name = "RAVI"
print(name.lower())
```

Output

```
ravi
```

Note

The upper() and lower() methods in Python work only on **alphabetic characters** and on **special characters** (like punctuation marks, symbols, digits, etc.), these methods do **not have any effect**.

For example:

PYTHON

Your Input

Your Input

Your Input

Your Input

Your Input

```python
s = "Hello, John! 123"
print(s.upper())  # Output: "HELLO, JOHN! 123"
print(s.lower())  # Output: "hello, john! 123"
```

